Electric Power Systems Research 211 (2022) 108447



Contents lists available at ScienceDirect

Electric Power Systems Research

journal homepage: www.elsevier.com/locate/epsr





Physics-informed neural networks for modelling power transformer's dynamic thermal behaviour☆,☆☆

Federica Bragone [a], Kateryna Morozovska [a,*], Patrik Hilber [a], Tor Laneryd [b], Michele Luvisotto [b]

[a] Department of Electrical Energy Engineering, KTH Royal Institute of Technology, Stockholm, Sweden
[b] Hitachi Energy Research, Västerås, Sweden

ARTICLE INFO

Keywords:
PINNs
Power transformers
Thermal modelling

ABSTRACT

This paper focuses on the thermal modelling of power transformers using physics-informed neural networks (PINNs). PINNs are neural networks trained to consider the physical laws provided by the general nonlinear partial differential equations (PDEs). The PDE considered for the study of power transformer's thermal behaviour is the heat diffusion equation provided with boundary conditions given by the ambient temperature at the bottom and the top-oil temperature at the top. The model is one dimensional along the transformer height. The top-oil temperature and the transformer's temperature distribution are estimated using field measurements of ambient temperature, top-oil temperature and the load factor. The measurements from a real transformer provide more realistic solution, but also an additional challenge. The Finite Volume Method (FVM) is used to calculate the solution of the equation and further to benchmark the predictions obtained by PINNs. The results obtained by PINNs for estimating the top-oil temperature and the transformer's thermal distribution show high accuracy and almost exactly mimic FVM solution.

1. Introduction

Power transformers are key power system components, which are connecting generation, transmission and distribution. In order to improve the efficiency of transformer design and operation, it is important to look for novel methods which can offer lesser computational intensity and more generalizable results. Dynamic thermal modelling is fundamental to study the thermal behaviour of power transformers. Various models to monitor the temperature in a transformer by estimating its top-oil temperature are used, including the IEC 60076-7 2nd edition model [1], the IEEE models [2], and models that consider a correspondent thermal circuit [3–5].

In recent years, the expansion of machine learning (ML) has contributed to different scientific fields. In particular, in the last thirty years, ML has started to solve differential equations. First, a neural algorithm to solve differential equations was proposed [6] and further developed in following studies implementing artificial neural networks (ANNs) [7,8] and deep learning [9]. Finally, a seminal paper in the area is on Physics-Informed Neural Networks (PINNs) by Raissi et al. [10]. The idea of PINNs consists of providing the neural networks with physical knowledge stored in the general nonlinear PDEs.

There are different examples of machine learning applied to this new approach using computational physics. They include support vector machines, Gaussian processes, random forests and more algorithms. However, the idea of treating neural networks not purely just as "black boxes" but as something that can give some understanding of the function that is approximating is very original.

Considerable attention to the topic of PINNs is given in the last years. In particular, studies treat the wave equation, KdV and KdV–Burger equations, all in 1D [11], the swing equation both for the forward and the inverse problem [12], the Poisson equation and the 1D diffusion equation [13], the 1D advection–diffusion equation [14], the heat transfer equation 1D [15]. Other applications include the Navier–Stokes equations in 2D for in-compressible laminar flows [16], the 2D Laplace equation and the 2D heat equation for aerospace applications [17] and many more. Therefore, the topic can cover various areas, and its use can make a difference within the physical and mathematical world.

☆ Authors thank SweGrids, Sweden and Energimyndigheten, Sweden for project sponsorship. We would like to recognize contribution of Ola Ivarsson and Claes Ahlrot who are with E.On Energidistribution AB, Sweden for guidance through the project and sponsorship of our research. Authors are also thankful to Erik Lejerskog and Ellevio AB, Sweden for project sponsorship and support.

☆☆ Submitted to the 22nd Power Systems Computation Conference (PSCC 2022).

* Corresponding author.
E-mail addresses: bragone@kth.se (F. Bragone), kmor@kth.se (K. Morozovska), tor.laneryd@hitachi-powergrids.com (T. Laneryd), michele.luvisotto@hitachi-powergrids.com (M. Luvisotto).

https://doi.org/10.1016/j.epsr.2022.108447
Received 11 September 2021; Received in revised form 7 April 2022; Accepted 2 July 2022
Available online 13 July 2022
0378-7796/© 2022 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

The PDE utilized for the study is the one-dimensional heat diffusion equation. The diffusion equation is already used for other studies using PINNs. An example is given by [15], where the heat source is not included for simplification, and different boundary conditions have been considered. A further step of the present work, and yet the hardest, is to use the heat diffusion equation with a heat source, which has been neglected for simplification in other works.

Moreover, different libraries have been created for this type of problems. DeepXDE [18] is an example that aims to solve time-dependent PDEs using just their initial conditions as an input. DeepXDE library works with TensorFlow [19]. For the same purpose, the Python package called SciANN [20] has been developed, making use of TensorFlow and Keras packages. However, the use of these libraries is out of the scope of the presented study. An insightful aspect that will be one of the main targets of implementing the PINN algorithm is given by [21], where the physics-informed approach is applied to the traffic density reconstruction; however, it also describes an investigation applicable to our case. It introduces the "first-order physics" term, which is a term that also includes measurements in its implementation. In our problem, the physical function also depends on measurements given by the load loss P_K and the ambient temperature T_a, both dependent on time and provided by the data.

This study uses PINNs as a method for estimating the top-oil temperature and evaluating the overall power transformer's thermal distribution. The PINNs model in this study is following the model proposed in [10], which is focusing on solving two problem types of several well-known PDEs: data-driven solution of PDEs and data-driven discovery of PDEs. The first method in [10] is dealing with finding the solution of PDEs in question, while the second method aims to learn the best parameters for describing the observed data.

2. Methodology

The model developed for solving the heat diffusion equation is based on the original work by Raissi et al. in [10], which is also available at GitHub repository https://github.com/maziarraissi/PINNs. The proposed changes include adapting the model to solve the PDEs of the heat diffusion equation as well as investigating hyperparameters to tune into the neural network (NN).

The general idea of PINNs is outlined in Algorithm 1.

Algorithm 1 The PINN algorithm for PDEs

Step 1: Construct a neural network with inputs x and t and parameters (w, b) corresponding to the weights and bias of the network.
Step 2: Define the two training sets for the equation and the boundary conditions.
Step 3: Define the loss function MSE as the sum of the mean squared error of both the equation and the boundary conditions.
Step 4: Train the neural network minimizing the loss function.

2.1. Problem description

The general form of the heat diffusion equation in 1D, as expressed by [22], is given by the following:

$$k\frac{\partial^2 T}{\partial x^2} + q = \rho c_p \frac{\partial T}{\partial t} \quad \Rightarrow \quad \frac{\partial^2 T}{\partial x^2} + \frac{q}{k} = \frac{1}{\alpha}\frac{\partial T}{\partial t}. \tag{1}$$

The term q in (1) describes a heat source, which is shown in Fig. 1 with horizontal arrows. The heat source is given by:

$$q = q(x,t) = P_0 + P_K(t) - h(T(x,t) - T_a(t)). \tag{2}$$

The problem aims to model the transformer vertically along its height H, as shown in Fig. 1.

In Eq. (2), the terms P_0 and $P_K(t)$ define the losses present in the transformers, particularly the no-load and load losses. In particular, $P_K(t) \sim K^2 \cdot \mu$, where K is the load factor and μ is the rated load loss. Moreover, the term h represents the convective heat transfer coefficient, which characterizes the heat transfer as a result of convection.

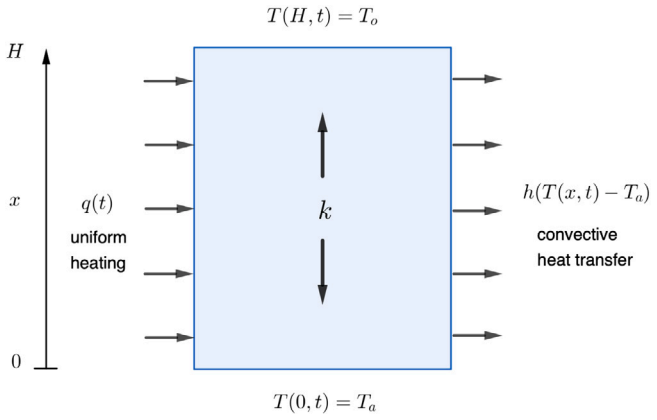


Fig. 1. Representation of the transformer with corresponding parameters of the heat diffusion equation model.

2.2. Data-driven solution of the heat diffusion equation

Consider the heat diffusion equation as described in (1); we now give a simplification of the notation by using $T(x,t) = u(x,t)$ and by implementing the subscript notation that can be written as:

$$\begin{aligned} &\rho c_p u_t - k u_{xx} - (P_0 + P_K - h(u - T_a)) = 0, \\ &u(0,t) = T_a, \\ &u(H,t) = T_o, \quad x \in [0, H] \quad \text{and} \quad t \in [0, n], \end{aligned} \tag{3}$$

where H is the height of the transformer, which is $H = 1$, and n represents the number of data points in time taken into consideration. The amount of time data points taken in the different assessments is $n = 100$.

To give a general idea about data-driven solution of PDEs as well as data-driven discovery of PDEs we consider the equation:

$$u_t + \mathcal{D}[u; \lambda] = 0, \quad x \in \Omega, \quad t \in [0, n]. \tag{4}$$

Eq. (4) is a generic description of a nonlinear partial differential equation, which includes the latent hidden solution $u(x,t)$, a parametrization given by the term λ of a nonlinear operator $\mathcal{D}[\cdot;\lambda]$, and a domain Ω, which is a subset of $\mathbb{R}^d$. Eq. (4) is the basis of different problems and systems.

First step is to obtain a data-driven solutions of partial differential equations, hence Eq. (4) is taken into consideration without the parameter λ. The left-hand-side of the equation is defined as $f(x,t)$ as follows:

$$f := u_t + \mathcal{D}[u]. \tag{5}$$

Given (5) we can define $f(x,t)$ as follows:

$$f := \rho c_p u_t - k u_{xx} - (P_0 + P_K - h(u - T_a)). \tag{6}$$

$u(x,t)$ is approximated by a neural network. There is a further step that we need to consider in this specific situation. The heat source term shown in Eq. (2) has parameters dependent on space and time. Indeed, the solution $u(x,t)$ is present, as well as the load factor $P_K(t)$ and the ambient temperature $T_a(t)$. Therefore, the function f in (6) depends directly on measurements, since we have data for P_K and T_a. The problem of data-driven discovery of partial differential equations instead applies a similar idea as the problem of finding a solution of the PDEs however, it uses Eq. (4) as it is, with the parameter λ included. The important thing to underline in this type of problems is that the parameter λ becomes the parameter of the physics-informed neural network $f(x,t)$.

Fig. 2 shows the model used for the data-driven solution of the heat diffusion equation. First, a neural network is trained with the boundary

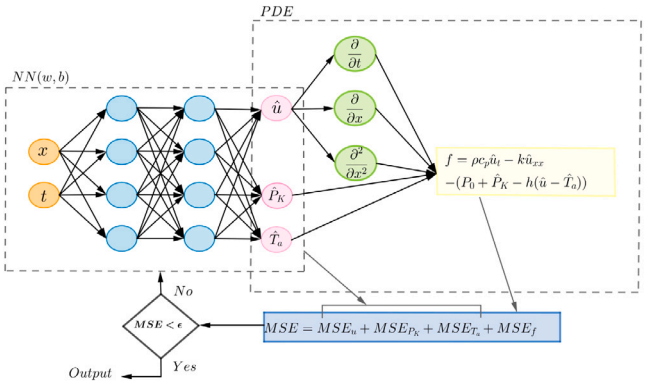


Fig. 2. Schematic representation of PINN for the problem.

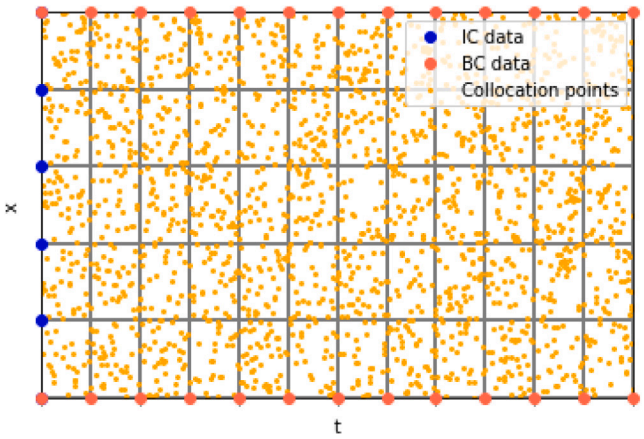


Fig. 3. Grid representing the training data for the model. The orange dots represents the boundary conditions data, the blue dots are the initial condition data (which are not available for this problem) and the yellow dots are the collocation points.

values for space and time to obtain an approximation of the solution of the temperature u and of the load loss P_K and the ambient temperature T_a. It is important to notice that while u depends on both space and time, P_K and T_a only change in time and remain constant in space. These approximations are consequently used to calculate the overall solution of the heat diffusion equation, shown in the yellow box.

$\hat{P}_K$ and $\hat{T}_a$ can be just inserted into $f(x,t)$, without needing further calculations. Because of the inclusion of these terms as outputs in NN, the overall loss function needs to consider their corresponding mean square error. Therefore, we obtain:

$$\text{MSE} = \text{MSE}_u + \text{MSE}_{P_K} + \text{MSE}_{T_a} + \text{MSE}_f, \tag{7}$$

where MSE_u and MSE_f are given by (8) and (9) respectively; MSE_{P_K} is expressed by (10) and MSE_{T_a} is expressed by (11).

$$\text{MSE}_u = \frac{1}{N_u}\sum_{i=1}^{N_u} |\hat{u}(x_u^i, t_u^i) - u^i|^2 \tag{8}$$

$$\text{MSE}_f = \frac{1}{N_f}\sum_{i=1}^{N_f} |f(x_f^i, t_f^i)|^2 \tag{9}$$

$$\text{MSE}_{P_K} = \frac{1}{N_u}\sum_{i=1}^{N_u} |\hat{P}_K(x_u^i, t_u^i) - P_K^i|^2 \tag{10}$$

$$\text{MSE}_{T_a} = \frac{1}{N_u}\sum_{i=1}^{N_u} |\hat{T}_a(x_u^i, t_u^i) - T_a^i|^2. \tag{11}$$

Here, $\{x_u^i, t_u^i, u^i, P_K^i, T_a^i\}_{i=1}^{N_u}$ represent the training data given by the boundary conditions and $\{x_f^i, t_f^i\}_{i=1}^{N_f}$ delineate the collocation points for $f(x,t)$. Consider the first 100 data points in time in the dataset; we have an array of 100 zeros and an array of 100 ones both corresponding to the coordinates in space and time of the boundary conditions. We end up having 200 points in space and 200 points in time. The orange dots at the bottom and the top of Fig. 3 represent these points given by the boundary conditions data. They are the overall number of training data, called N. Moreover, temperatures at the boundary points are also known because provided by the dataset by T_a and T_o. The temperatures are used to minimize MSE_u. We generate a random sample from N of size N_u. Therefore, N_u represents the number of generated random samples used as training points. If we pick $N_u = 50$, we will have 50 training points in space in total between the two boundary conditions and their corresponding locations in time. Similarly, we will have 50 training points for the corresponding temperatures T_a and T_o, and load factor P_K for those values of space and time.

We are taking a smaller size random sample of the training points to show that PINNs can perform well without using a large amount of training data. Different sizes of N_u will be experimented with to obtain results, and more about them can be found in Section 2.3, when talking about parameters tuning.

As it can be seen in Fig. 2, the approximation of the solution $\hat{u}$ now is taken as input and used to calculate its derivatives with respect to the collocation points just generated by exploiting automatic differentiation (AD) [23]. The model is created using the Python open-source library TensorFlow [19]. The `tf.gradients` is utilized to calculate the derivatives of the approximated solution $\hat{u}$, obtained by the previous neural network, with respect to x and t at selected collocation points. The collocation points are points in the entire time/space domain, and they are generated using the Latin hypercube sampling [24]. This sampling is a statistical method that creates samples of values in a quasi-random way. It is often used for simulations of models because of its simplicity and capacity to handle multidimensional parameters. The yellow points in Fig. 3 illustrates how the collocation points are taken in the considered domain. N_f expresses the number of collocation points for $f(x,t)$. Increasing N_f does not imply adding new data, but it simply means evaluating the equation at the corresponding locations. Therefore, increasing N_f has a lower cost than increasing N_u.

Consequently, the loss function for $f(x,t)$, MSE_f, is calculated and added to the training losses MSE_u, MSE_{P_K} and MSE_{T_a} to obtain an overall loss defined in (7).

Table 1 shows the parameters with corresponding values and units. The effective thermal conductivity k in the model is set to approximate heat convection. The convective heat transfer coefficient h would need to be multiplied by a particular factor instead, since we are solving the 1D heat diffusion equation with loss generated and distributed over the height of the transformer. It follows then that k and h are assigned with larger values than expected to describe the physics beyond the heat diffusion equation. For this reason, the model does not capture the strictly increasing temperature along the transformer's height typical of the buoyancy of the oil. The height considered is of 1 meter, which does not resemble the real height of a power transformer, but it was taken as a simplification for the implementation of the model. Another consideration that needs to be made, when implementing the PINN algorithm for the present problem, is parameter scaling. Given data from Table 1, the parameters of the heat diffusion equation are quite large, and since the NN would have some difficulties optimizing the loss given such large numbers, some scaling is required. In particular, the parameters μ, P_0, c_p, ρ, k and h are scaled by a factor of 1000, while their units change accordingly, the overall performance is not affected. The function f then becomes:

$$\begin{aligned} f &= 900 \cdot 2000u_t - 50u_{xx} - (15000 + 83000K^2 - \\ &\quad - 1000(u - T_a)) \Rightarrow f = 0.9 \cdot 2u_t - 0.05u_{xx} - \\ &\quad - (15 + 83 \cdot K^2 - 1 \cdot (u - T_a)) \end{aligned} \tag{12}$$

Table 1
Parameters and corresponding values of the heat diffusion equation.

Parameter	Value	Unit
Effective thermal conductivity k	50	(W/m K)
Specific heat capacity c_p	2000	(J/kg K)
Density ρ	900	(kg/m^3)
Convective heat transfer coefficient h	1000	W/m^2 K
No-load loss P_0	15000	W
Rated load loss μ	83000	W
Height H	1	m

Table 2
Parameters and corresponding values for the method.

Parameter	Values
N_u	50, 100, 150
N_f	2000, 5000, 10000
Hidden layers	2, 4, 6
Neurons	10, 20, 50, 100
Activation function	tanh
Optimizer	L-BFGS-B

2.3. Hyperparameter tuning

It is important to consider hyperparameters for achieving the desired performance in an artificial neural network. Moreover, the numbers of training points and collocation points evaluating the solutions are fundamental for the overall performance. Table 2 outlines the parameters and the values used for evaluating the problem. All the possible combinations of the values reported in the table are used to gather results. Choosing the most suitable parameters can make a significant improvement of the model's accuracy.

In Table 2 values for the number of hidden layers and neurons are selected to show some basic neural networks structure. It is analysed how the wideness and the depth of the architecture affect the precision of the predictions.

3. Results

The results have been run using the first 100 h of the dataset and its corresponding measurements for the ambient temperature T_a, top-oil temperature T_o and load factor K. As already mentioned in Table 2, the number of hidden layers considered for the neural network are 2, 4, 6 since additional ones did not seem to be necessary given the amount of training data. Moreover, the number of neurons used is 10, 20, 50, 100. The activation function utilized is the hyperbolic tangent $\tanh(x)$. For the training points, N_u, we take 50, 100 and 150 out of the 200 overall training data available. Recall that using 100 h data points in time means 100 data points for T_a and 100 data points for T_o to use for training. Finally, for the collocation points, N_f, we considered 2000, 5000 and 10,000.

The results for all the tests performed in this study are presented further. The results include the investigation of the best hyperparameters according to Table 2. The PINN method is compared to the finite volume method (FVM), which is used as a benchmark to test the overall model performance. The FVM results are obtained using 50 nodes for the space discretization and 100 points for the time discretization (100 h). The lowest relative $\mathbb{L}_2$ errors for both the overall solution predictions and the predicted top-oil temperatures are 1.45e−01 and 2.59e−02, respectively.

The best result is obtained using $N_u = 100$, $N_f = 10000$ and a neural network architecture of 4 hidden layers with 50 neurons each with the hyperbolic tangent, $\tanh(x)$, as activation function and the L-BFGS-B as optimizer. Table 3 shows all the combinations for the various numbers of training points N_u and collocation points N_f for the given network architecture. The lowest error is 1.45e−01 and is highlighted in bold in the table and provided by $N_u = 100$ and $N_f = 10000$. If we look at the top-oil temperature error in Table 4 for the same values of training points and collocation points, we get a value of 2.59e−02. The value is not the lowest of the table; however, considering it with the solutions' overall approximation error of Table 3, they give the best combination for accuracy.

Table 3
A relative $\mathbb{L}_2$ error between the solution given by FVM and the predicted solution $\hat{u}(x,t)$ for variable N_u and N_f. The NN consists of 4 layers, 50 neurons each. The time frame covers first 100 h of the dataset.

	$N_f = 2000$	$N_f = 5000$	$N_f = 10000$
$N_u = 50$	2.11e−01	1.94e−01	2.41e−01
$N_u = 100$	2.55e−01	2.50e−01	**1.45e − 01**
$N_u = 150$	2.03e−01	2.59e−01	1.57e−01

Table 4
A relative $\mathbb{L}_2$ error between T_o data and its predicted value $\hat{T}_o$ for variable of N_u and N_f. The NN consists of 4 layers, 50 neurons each. The time frame covers first 100 h of the dataset.

	$N_f = 2000$	$N_f = 5000$	$N_f = 10000$
$N_u = 50$	3.53e−02	5.05e−02	1.56e−01
$N_u = 100$	2.89e−02	6.06e−02	**2.59e − 02**
$N_u = 150$	5.75e−02	1.74e−02	3.26e−02

Table 5
A relative $\mathbb{L}_2$ error between FVM solution and the predicted solution $\hat{u}(x,t)$ for a different number of hidden layers and neurons. $N_u = 100$, $N_f = 10000$, the time frame covers first 100 h of the dataset.

Layers	Neurons			
	10	20	50	100
2	2.58e−01	1.55e−01	2.57e−01	2.66e−01
4	1.88e−01	1.68e−01	**1.45e − 01**	1.87e−01
6	3.44e−01	3.18e−01	1.32e−01	4.14e−01

Table 6
A relative $\mathbb{L}_2$ error between T_o data and its predicted value $\hat{T}_o$ for a different number of hidden layers and neurons. $N_u = 100$, $N_f = 10000$, the time frame covers first 100 h of the dataset.

Layers	Neurons			
	10	20	50	100
2	9.42e−02	9.16e−02	2.80e−01	3.67e−02
4	6.69e−02	5.50e−02	**2.59e − 02**	3.15e−02
6	2.46e−01	7.60e−02	3.07e−02	3.41e−02

Further analysis is given by Tables 5 and 6, where the number of training points and collocation points is fixed for $N_u = 100$ and $N_f = 10000$, respectively. The neural network architecture is evaluated by comparing the number of layers versus the number of neurons. Again, the relative $\mathbb{L}_2$ errors are reported for approximating the solution $\hat{u}(x,t)$ in Table 5, and the approximation of the top-oil temperature in Table 6. If we look at the two tables separately, for Table 5 we would pick the architecture with 6 hidden layers and 50 neurons each, since the error is the lowest and given by 1.32e−01. While for Table 6, we would pick 4 hidden layers and 50 neurons since the error is 2.59e−02. As above, we are looking at the overall performance that gives accurate results for both predictions. Therefore, the architecture considered is 4 hidden layers and 50 neurons since the approximated solution error is 1.45e−01.

The parameters giving the best results are 4 layers, 50 neurons, $N_u = 100$ and $N_f = 10000$. The number of iterations before convergence is 28705, while the training time is 7285.328 s, approximately 2 h on a laptop with a CPU frequency of 2.21 GHz and 4 cores. Fig. 4 shows the plot representing the solutions according to the above values.

The panel located at the top of Fig. 4 displays the predicted spatio-temporal solution $u(x,t)$. Moreover, the points at the bottom and top of the panel represent the location of the boundary conditions. The three bottom panels are a comparison between FVM solution (in blue)

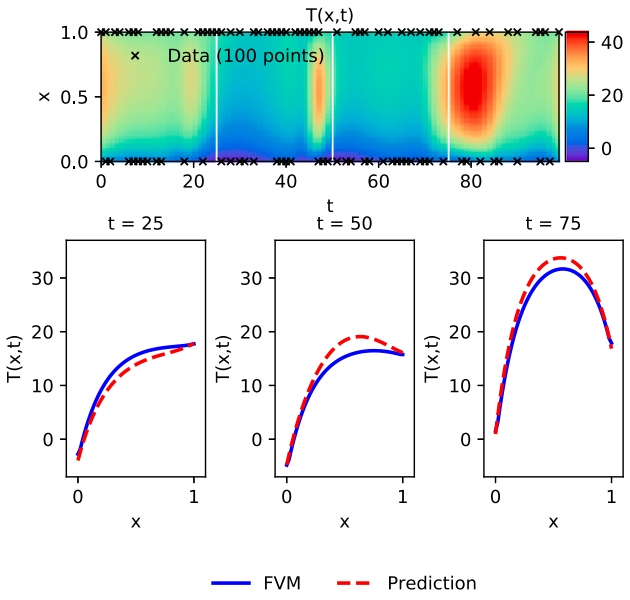


Fig. 4. Plot of the results obtained where 4 layers and 50 neurons each were used with $N_u = 100$ and $N_f = 10000$ over the time period 1–100. $T(x,t)$ represents the solution $u(x,t)$, x is the space, which is the height of the transformer, t is measured in hours.

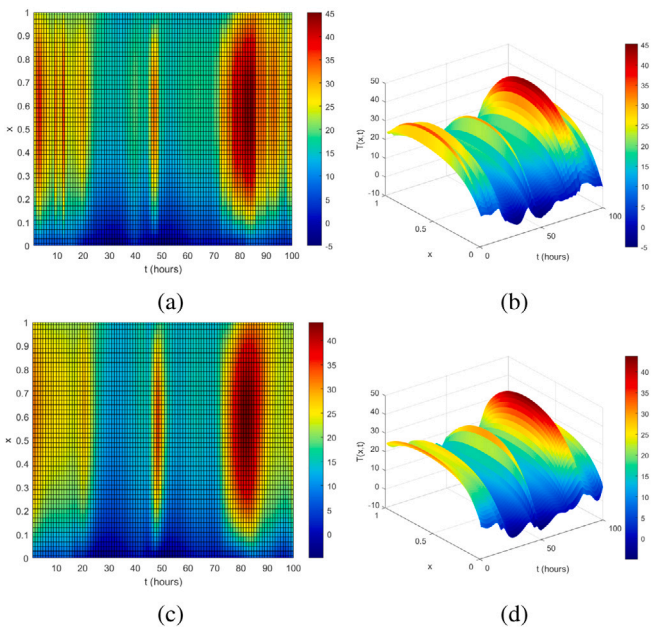


Fig. 5. Plots of the solutions obtained with FVM compared to the predicted solutions estimated by PINN for the first 100 h of the dataset. $T(x,t)$ represents the solution $u(x,t)$. (a): 2D plot of all the solutions $u(x,t)$ over the time considered with FVM. (b): 3D plot of all the solutions $u(x,t)$ over the time considered with FVM. (c): 2D plot of all the predicted solutions $\hat{u}(x,t)$ over the time considered with PINN. (d): 3D plot of all the predicted solutions $\hat{u}(x,t)$ over the time considered with PINN.

and PINN solution (red-dotted line). Three times have been taken into consideration, i.e. $t = 25$, $t = 50$ and $t = 75$. Ideally, the red-dotted line should match the blue line completely, assuming that FVM gives a solution as close as possible to the analytical solution. However, the results displayed already give a good performance overall and show that PINNs can give promising results given proper parameter tuning.

The accuracy can be seen in Fig. 5, where the FVM solution (Figs. 5(a) and 5(b)) is shown along with the predictions obtained by PINN (Figs. 5(c) and 5(d)). In particular, Fig. 5(a) represents the solution over a 2D plot of FVM, while Fig. 5(c) shows the corresponding 2D plot for the PINN predictions. Fig. 5(c) should mimic as close as possible Fig. 5(a). It is possible to notice similarities in the graphs, guaranteeing a good performance of the PINN model.

Generally, the results obtained from all the runs give fundamental insight into how the parameters affect the model's accuracy. It should be mentioned that a small number of neurons gave a faster convergence, but a lower accuracy. The more neurons and layers are augmented, the more the training time and the number of iterations increase. The errors collected for the approximated solution $\hat{u}(x,t)$ are all relatively low in a range between 1.28e+00 and 1.32e−01. Therefore, further improvements could lower these values and obtain a more accurate model. However, better performance can be noticed for approximating the top-oil temperature $\hat{T}_o$. The relative $\mathbb{L}_2$ error values are between 3.58e−01 and 1.30e−02. A more drastic drop in the error shows more accurate predictions than the overall approximation of the temperature distribution. This is an important discovery since the main goal is to predict the top-oil temperature.

4. Conclusions

This work aims to predict the top-oil temperature in a transformer using physics-informed neural networks while also computing the distribution of the temperature in the transformer.

The results obtained provide further considerations to carry out. The insertion of the load factor P_K and the ambient temperature T_a is complex because of their dependency on time. This aspect introduces difficulties but it also guarantees more relevance to the overall study. Overcoming the issue by approximating the values for P_K and T_a in the neural network gives a good solution. In the results FVM is used as a benchmark to compare the solutions obtained by PINNs. In general, when a numerical scheme is provided with the right amount of data like in this case with the given boundary conditions, then its performance is overall accurate. PINNs cannot outperform FVM given the quantity and the type of data used. However, PINNs are able to leverage existing measurements, and for this reason in more complex scenarios PINNs are more likely to work better compared to numerical methods.

One of the advantages of PINNs is being good at dealing with both large and small datasets, unlike artificial neural networks which need a large quantity of data. One of the main drawbacks is the extensive computation time.

This study showed promising results in applying PINNs in the dynamic thermal modelling of power transformers. The heat diffusion equation is a simplification of the physical behaviour of a power transformer. However, this equation gives the basis to create a more complex and accurate model that resembles the physical processes in the dynamic thermal behaviour of power transformers. Future work could include various changes in the model that will lead to substantial improvements in the performance.

CRediT authorship contribution statement

Federica Bragone: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft. **Kateryna Morozovska:** Conceptualization, Data curation, Resources, Funding acquisition, Supervision, Writing – original draft. **Patrik Hilber:** Project administration, Resources. **Tor Laneryd:** Conceptualization, Data curation, Resources, Funding acquisition, Supervision, Writing – review & editing. **Michele Luvisotto:** Conceptualization, Data curation, Resources, Funding acquisition, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] IEC, Power transformers - Part 7: Loading guide for oil-immersed power transformers, IEC Stand. 60076 (7:2018 RLV) (2018).

[2] IEEE, Guide for loading mineral-oil-immersed transformers and step- voltage regulators, IEEE Stand. (2011) C57.91.

[3] D. Susa, M. Lehtonen, H. Nordman, Dynamic thermal modelling of power transformers, IEEE Trans. Power Deliv. 20 (1) (2005) 197–204.

[4] D. Susa, M. Lehtonen, Dynamic thermal modeling of power transformers: further development-Part I, IEEE Trans. Power Deliv. 21 (4) (2006) 1961–1970.

[5] D. Susa, M. Lehtonen, Dynamic thermal modelling of power transformers: further development-Part II, IEEE Trans. Power Deliv. 21 (4) (2006) 1971–1980.

[6] H. Lee, I.S. Kang, Neural algorithm for solving differential equations, J. Comput. Phys. 91 (1) (1990) 110–131.

[7] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.

[8] I.E. Lagaris, A.C. Likas, D.G. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, IEEE Trans. Neural Netw. 11 (5) (2000) 1041–1049.

[9] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, J. Comput. Phys. 375 (2018) 1339–1364.

[10] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.

[11] Y. Guo, X. Cao, B. Liu, M. Gao, Solving partial differential equations using deep learning and physical constraints, Appl. Sci. 10 (17) (2020) 5917.

[12] G.S. Misyris, A. Venzke, S. Chatzivasileiadis, Physics-informed neural networks for power systems, in: 2020 IEEE Power & Energy Society General Meeting (PESGM), IEEE, 2020, pp. 1–5.

[13] Q. Zhang, Y. Chen, Z. Yang, Data-driven solutions and discoveries in mechanics using physics informed neural network, 2020.

[14] E. Wasei, Investigating physics-informed neural networks for solving PDEs, 2020.

[15] N. Zobeiry, K.D. Humfeld, A physics-informed machine learning approach for solving heat transfer equation in advanced manufacturing and engineering applications, Eng. Appl. Artif. Intell. 101 (2021) 104232.

[16] C. Rao, H. Sun, Y. Liu, Physics-informed deep learning for incompressible laminar flows, Theoret. Appl. Mech. Lett. 10 (3) (2020) 207–212.

[17] H. Karali, U.M. Demirezen, M.A. Yukselen, G. Inalhan, A novel physics informed deep learning method for simulation-based modelling, in: AIAA Scitech 2021 Forum, 2021, p. 0177.

[18] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, SIAM Rev. 63 (1) (2021) 208–228.

[19] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al., Tensorflow: A system for large-scale machine learning, in: 12th $\{USENIX\}$ Symposium on Operating Systems Design and Implementation ($\{OSDI\}$ 16), 2016, pp. 265–283.

[20] E. Haghighat, R. Juanes, SciANN: A keras wrapper for scientific computations and physics-informed deep learning using artificial neural networks, 2020, arXiv preprint arXiv:2005.08803.

[21] M. Barreau, M. Aguiar, J. Liu, K.H. Johansson, Physics-informed learning for identification and state reconstruction of traffic density, 2021, arXiv preprint arXiv:2103.13852.

[22] F.P. Incropera, A.S. Lavine, T.L. Bergman, D.P. DeWitt, Fundamentals of Heat and Mass Transfer, Wiley, 2007.

[23] A.G. Baydin, B.A. Pearlmutter, A.A. Radul, J.M. Siskind, Automatic differentiation in machine learning: a survey, J. Mach. Learn. Res. 18 (2018).

[24] M.D. McKay, R.J. Beckman, W.J. Conover, A comparison of three methods for selecting values of input variables in the analysis of output from a computer code, Technometrics 42 (1) (2000) 55–61.